

Esercizi su specifiche ASM

Esercizio 1

Modellare con le ASM il funzionamento di un sistema di imbottigliamento.

Un sensore esterno segnala la presenza di una bottiglia vuota nella sede apposita.

La bottiglia viene quindi riempita ed un sensore segnala il completo riempimento della bottiglia.

La bottiglia viene quindi tappata e poi rimossa dalla sede.

Un eventuale errore del dispositivo, pone l'intero sistema in stato di errore.

Modello

Segnatura:

bottigliaIN: Boolean monitorata

bottigliaFULL: Boolean monitorata

bottigliaOUT: Boolean monitorata

stato: {attesa, imbottigliamento, tappare}
controllata

error : Boolean monitorata

Stato iniziale:

stato = attesa

main_rule=

```
if not(error) then
```

par

```
if stato = attesa ^ bottigliaIN
```

```
then stato:= imbottigliamento
```

```
if stato = imbottigliamento ^ bottigliaFULL
```

```
then stato:= tappare
```

```
if stato = tappare ^ bottigliaOUT
```

```
then stato:= attesa
```

Nota: è possibile aggiungere una regola

if error then skip

come placemaker per l'error handling

ma la run della macchina non cambia

Esercizio 2

Specificare un modello ASM per il seguente funzionamento di una struttura dati “queue”. All’inizio la coda è vuota.

Gli eventi interessanti sono:

- *insert_rear* per inserire un elemento in coda
- *delete_front* per cancellare un elemento dalla testa della coda

Una variabile globale *size* rappresenta l’effettiva lunghezza della coda. Essa vale 0 quando la coda è vuota, e *k* (una costante globale) quando è piena.

Modello – prima versione più astratta: gestione solo di size

Signature:

```
insert: Boolean, monitored
delete: Boolean, monitored
size: Integer, controlled
k: Integer, static
codapiena: Boolean, derived
codavuota: Boolean, derived
outmsg: String controlled
```

Definitions:

```
function k = 10
function codapiena == (size = k)
function codavuota == (size = 0)
```

```
macro_rule insert_rear =
  if insert and not delete then
    if not codapiena
      then size:= size +1
    else outmsg:= "coda piena"
```

```
macro_rule delete_front =
  if delete and not insert
  then
    if not codavuota
      then size:= size -1
    else outmsg:= "coda vuota"
```

```
main_rule =
par
  insert_rear[]
  delete_front[]
endpar
```

```
S0 init state:
  size = 0
```

NOTA: le run che hanno stati in cui simultaneamente sono vere sia insert che delete possono essere scartate aggiungendo al modello l'invariante: (insert xor delete).

Aggiungendo l'invariante, nelle regole insert_rear e delete_front si possono eliminare i controlli in rosso.

Modello – versione meno astratta: gestione dell'elemento inserito

Signature:

```
insert: Integer → Boolean, monitored
delete: Boolean, monitored
size: Integer, controlled
k: Integer, static
q: Integer → Integer, controlled  \\ Modella una data posizione dell'array
codapiena: Boolean, derived
codavuota: Boolean, derived
outmsg: String controlled
```

Definitions:

```
function k = 10
function codapiena == (size = k)
function codavuota == (size = 0)
```

macro_rule insert_rear =

```
if insert(x) and not delete then
  if not codapiena
  then
    par
      q(size) := x
      size := size + 1
    else outmsg := "coda piena"
```

macro_rule delete_front =

```
if delete and (forall i in Integer | not insert(i))
then
  if not codavuota
  then
    par
      size := size - 1
      outmsg := q(size)
    else outmsg := "coda vuota"
```

main_rule =

```
par
  insert_rear[]
  delete_front[]
endpar
```

S0 init state:

```
function size = 0
\\ la funzione q(i) non inizializzata ha valore undef per ogni i in Integer
\\ anche in questo modello vale il discorso delle monitorate insert(x) e delete
```

NOTA: la sintassi dl modello è libera e non fedele ad AsmetaL

Esercizio 3

Specificare un modello ASM per il seguente funzionamento di sistema. Il sistema può trovarsi in modalità di funzionamento *normale* ed in modalità di *manutenzione*.

In modalità manutenzione vengono svolte due attività parallele: un'attività di testing dei devices del sistema e successiva fase di diagnosi del sistema completo, ed un'attività di controllo (in parallelo) dei tre comandi del sistema. Terminate le attività relative alla fase di manutenzione, il sistema ritorna in modalità di funzionamento normale.

Modello

Segnatura:

needManutenzione : Boolean **monitorata**

FASE={normale, manutenzione} **dominio**

fase: FASE **controllata**

r_testing = skip

r_check1 = skip

r_diagnosi = skip

r_check2 = skip

r_check3 = skip

r_act1[] =

seq

r_testing[]

r_diagnosi[]

r_act2[] =

par

r_check1[]

r_check2[]

r_check3[]

r_normale =

if fase=normale $\wedge$ needManutenzione

then fase:= manutenzione

r_manutenzione =

if fase = manutenzione

then

par

r_act1[]

r_act2[]

fase:= normale

main_rule=

par

r_normale[]

r_manutenzione[]

Stato iniziale:

fase = normale

Esercizio 4

Specificare un modello ASM del funzionamento di un robot per la pulizia di pavimenti. Il robot viene attivato tramite un bottone I/O. Ha un sensore *dirty* che è indice di pulizia; se *dirty* è rosso il robot inizia a pulire, se è verde il robot si ferma.

Il robot è anche dotato di un sensore *tank* che indica la quantità di sporco dentro il serbatoio di accumulo. Tank ha valore verde se il serbatoio non è pieno, altrimenti la spia diventa rossa. Se il serbatoio è pieno, il robot si ferma e riparte solo a serbatoio svuotato.

Un errore dei sensori mette il dispositivo in stato di errore.

Modello

SEGNATURA:

```
enum domain ColorLight = {VERDE, ROSSO}  
dirty: ColorLight, monitored  
thank: ColorLight, monitored  
error: Bool, monitorata  
i_o: Bool, monitorata  
stato: {fermo, attivo}
```

REGOLE:

```
main rule main =
```

```
  par
```

```
    if stato = fermo AND i_o AND dirty then
```

```
      par
```

```
        stato:= attivo
```

```
    if not(i_o) OR dirty = verde OR thank = rosso
```

```
    then stato:= fermo
```

```
    if not (error) AND stato= attivo then
```

```
      par
```

```
        r_pulisci
```

```
        r_svuota
```

```
macro rule r_pulisci =
```

```
  if dirty = rosso AND thank = verde
```

```
  then act1[] //funzione per la pulizia
```

```
macro rule r_svuota =
```

```
if (thank = rosso) then
```

```
  par
```

```
    act2[] //funzione per lo svuotamento del serbatoio
```

```
    stato:= attivo
```

```
macro rule act2 = skip
```

```
macro rule act1 = skip
```

```
Stato iniziale:
```

```
stato= fermo
```

Se *tank* fosse dichiarata shared potrebbe essere aggiornata dalla macchina dopo lo svuotamento del servatorio:

```
macro rule r_svuota =
```

```
  if (thank = rosso) then
```

```
    par
```

```
      seq
```

```
        act2[]
```

```
        tank:= verde
```

```
        stato:= attivo
```

Modello

separazione dei concetti in mail rule:

REGOLE:

main rule main =

par

start[]

ferma[]

inAzione[]

macro rule start =

if stato = stato = fermo AND i_o AND dirty then

par

stato:= attivo

macro rule ferma =

if not(i_o) OR dirty = verde OR thank = rosso

then stato:= fermo

macro rule inAzione =

if not (error) AND stato= attivo then

par

r_pulisci

r_svuota

Esercizio 5

Modellare con le ASM il funzionamento di un dispositivo di stampa. Il dispositivo può essere in stato di funzionamento normale, degradato e di stop. In stato di normale funzionamento, il dispositivo serve per stampare file ricevuti dalla rete, passare allo scanner documenti, fotocopiare documenti. Le diverse funzionalità sono gestite tramite opportuni segnali. In caso di simultaneità di segnali, la stampa di documenti in rete ha priorità rispetto alla fotocopia e poi allo scanner. Se il dispositivo lavora in stato di funzionamento ridotto, è solo possibile la stampa. Quando è in stato di stop, un messaggio sul display segnala che il dispositivo è fuori servizio.

Modello

Segnatura:

state: {norm, deg, stop} monitorata

print: Boolean monitorata

scan: Boolean monitorata

copy: Boolean monitorata

outMsg: String controllata

macro_rule normMng=

seq

if print then doPrint[]

if copy then doCopy[]

if scan then doScan[]

endseq

main_rule=

par

if state = norm then

normMng[]

if state = deg AND print then

doPrint[]

if state = stop then

outMsg := "device unreachable"

endpar

S0 init state:

outMsg = ""

Esercizio 6

Specificare un modello ASM per il funzionamento del seguente sistema di parcheggio. Il parcheggio è dotato di due telecamere, una installata all'ingresso del parcheggio ed una all'uscita dal parcheggio, e di un semaforo che segnala, se verde, la disponibilità di posti liberi nel parcheggio. Per ogni auto che entra, la telecamera ne segnala l'accesso al controllore che tiene informazione del numero delle auto parcheggiate. Quando i posti si sono esauriti, il controllore mette a rosso il semaforo. Appena ritorna la disponibilità di posti, il semaforo viene messo a verde.

Modello

signature:

monitored cam_IN : boolean

monitored cam_OUT : boolean

derived semaforo : Boolean

\\ semaforo vale 1 se VERDE e vale 0 se ROSSO

controlled posti : integer

static max_posti : integer

definitions:

function max_posti = 100

function semaforo == posti <= max_posti

macro rule in_arrivo() =

if (cam_IN and semaforo) then posti = posti + 1

macro rule in_uscita() =

if cam_OUT then posti = posti - 1

macro rule main =

par

in_arrivo[]

in_uscita[]

endpar

default init s0:

posti = 0

Modello

Se cam_IN e cam_OUT sono vere simultaneamente si ha un update inconsistente su posti.

Ci sono 4 modi per evitarlo:

1. Aggiungere al modello l'invariante: (cam_IN xor cam_OUT) che permette di scartare tutte le run che violano l'invariante
2. Sequenzializzare il controllo delle telecamere, in tal caso la main rule diventa:
macro rule main =
 seq
 in_arrivo[]
 in_uscita[]
 endpar
3. Escludere nella guardia delle regole che i due eventi possono accadere insieme
 - if (cam_IN and semaforo and not cam_out) in macro rule in_arrivo()
 - If (not cam_IN and cam_out) in macro rule in_uscita()

Esercizio 7

Modellare con le ASM il comportamento del seguente sistema di condizionamento. Il sistema è composto da un controllore e da una condizionatore.

- In fase giorno, il controllore invia il comando di accensione al condizionatore se la temperatura ambiente supera il valore settato dall'utente. Per default la temperatura ambiente è fissata a 26°, ma l'utente può cambiare a piacere questo valore. Il condizionatore si spegne quando arriva in temperatura.
- In fase notte, il controllore invia al condizionatore un comando di accensione se la temperatura è superiore ad un valore minimo. Tale valore può essere stabilito a piacere dall'utente. Per default, esso è fissato a 28°.
- Un eventuale guasto del condizionatore, mette l'intero sistema in una situazione di guasto.

Modello

Signature:

enum domain StatoCondizionatore = {ON, OFF}

enum domain Fasi = {GIORNO, NOTTE}

monitored fase: Fasi

monitored cambioTemperatura : Boolean

monitored temperaturaAmbiente : Integer

monitored nuovaTemperatura : Integer

controlled temperaturaAmbienteGiorno : Integer

controlled temperaturaAmbienteNotte : Integer

controlled statoCondizionatore : StatoCondizionatore

main rule r_Main =

par

r_controlloNotte[]

r_controlloGiorno[]

default init:

temperaturaAmbienteGiorno = 26

temperaturaAmbienteNotte = 28

statoCondizionatore = OFF

r_controlloGiorno =

if fase = GIORNO then

par

if cambioTemperatura

then temperaturaAmbienteGiorno := nuovaTemperatura

if temperaturaAmbiente > temperaturaAmbienteGiorno

then statoCondizionatore := ON

if temperaturaAmbiente <= temperaturaAmbienteGiorno

then statoCondizionatore := OFF

r_controlloNotte =

if fase = NOTTE then

par

if cambioTemperatura

then temperaturaAmbienteNotte := nuovaTemperatura

if temperaturaAmbiente > temperaturaAmbienteNotte

then statoCondizionatore := ON

if temperaturaAmbiente <= temperaturaAmbienteNotte

then statoCondizionatore := OFF

Modello (vers 2)

Signature:

enum domain StatoCondizionatore = {ON, OFF}

enum domain Fasi = {GIORNO, NOTTE}

monitored fase: Fasi

~~monitored cambioTemperatura : Boolean~~

monitored temperaturaAmbiente : Integer

monitored nuovaTemperatura : Integer //x nuovo valore da utente

controlled temperaturaAmbienteGiorno : Integer

controlled temperaturaAmbienteNotte : Integer

controlled statoCondizionatore : StatoCondizionatore

main rule r_Main =

par

r_controlloNotte[]

r_controlloGiorno[]

default init:

temperaturaAmbienteGiorno = 26

temperaturaAmbienteNotte = 28

statoCondizionatore = OFF

r_controlloGiorno =

if fase = GIORNO then

par

if nuovaTemperatura

then temperaturaAmbienteGiorno := nuovaTemperatura

if temperaturaAmbiente > temperaturaAmbienteGiorno

then statoCondizionatore := ON

if temperaturaAmbiente <= temperaturaAmbienteGiorno

then statoCondizionatore := OFF

r_controlloNotte =

if fase = NOTTE then

par

if nuovaTemperatura

then temperaturaAmbienteNotte := nuovaTemperatura

if temperaturaAmbiente > temperaturaAmbienteNotte

then statoCondizionatore := ON

if temperaturaAmbiente <= temperaturaAmbienteNotte

then statoCondizionatore := OFF

Esercizio 8

Specificare un modello ASM per modellare il comportamento del seguente sistema di illuminazione dell'atrio di una casa.

Se è giorno, la luce non viene accesa. Se è notte, la luce viene accesa per 10 minuti appena viene aperto il cancello di casa.

Modello

signature:

monitored passed_10: Boolean

monitored day: Boolean

monitored cancello: Boolean

controlled luce: Boolean

main rule r_Main=

par

if not(day) then

par

if cancello then luce:=true

if luce=true AND passed_10

then luce:=false

if day then luce:=false

default init s0:

luce=false

Esercizio 9

Specificare un modello ASM per il seguente sistema di illuminazione automatica dei bagni del dipartimento. Due sensori, *sensore-luce* e *sensore-movimento*, sono coinvolti nel funzionamento. Il sensore-luce può essere attivato o disattivato. La sua attivazione è manuale. Se disattivato, le luci dei bagni vengono automaticamente accese quando il sensore-movimento rileva la presenza di una persona. La luce è spenta automaticamente quando il sensore non rileva più alcun movimento. Se il sensore-luce è attivo, permette di disattivare il funzionamento del sistema di illuminazione in *fase giorno*, mentre in *fase notte* rimane sempre attiva una luce di emergenza. In fase notte, se il sensore-movimento rileva la presenza di una persona nella zona bagni, le luci vengono automaticamente accese e così rimangono fino a che il sensore non rileva più alcun movimento. Quindi, la luce ritorna in modalità emergenza.

Esercizio 10 (multi agenti)

Si vuole modellare in ASM l'accesso ad una regione critica da parte di due o più processi.

L'accesso alla regione è coordinato dall'utilizzo di un *semaforo*:

solo un processo alla volta può trovarsi nella regione critica.

Un processo può trovarsi in quattro stati:

- *idle*: non vuole entrare nella regione critica;
 - *entering*: ha fatto richiesta di entrare nella regione critica e aspetta che il semaforo sia libero;
 - *critical*: è nella regione critica con il semaforo occupato (lo occupa lui quando entra);
 - *exiting*: sta uscendo dalla regione critica e libera il semaforo (lo libera solo quando esce da questo stato).
- Il modello ASM deve essere ad agenti;
 - l'utente, durante la simulazione, deve poter decidere se un processo vuole entrare/uscire dalla regione critica;
 - nello stato iniziale tutti i processi sono in idle ed il semaforo è libero.

Esercizio 10 (cont)

Invarianti per il simulatore AsmetaS

- due processi non possono mai essere contemporaneamente nella regione critica (per la versione con due processi);
- in generale, se ci sono n processi, al massimo un processo è nella regione critica.

Esercizio 11

La politica di schedulazione Shortest job next (SJN)¹ seleziona il processo in attesa che ha il più piccolo tempo d'esecuzione.

- lo scheduler deve essere un agente;
- i processi devono essere modellati come agenti;
- per ogni processo, bisogna memorizzare il suo stato (WAITING, RUNNING o DEAD) e il tempo che manca al suo completamento;
- la macchina deve memorizzare chi detiene il controllo della cpu (o un processo o lo scheduler).

Esercizio 11

- **Il programma di ogni processo è il seguente:**
- se non detiene il controllo della cpu non fa niente, altrimenti:
 - se il tempo d'esecuzione del processo è maggiore di 0, il tempo viene decrementato di un'unità;
 - se il tempo d'esecuzione del processo è uguale a 0, il processo diventa DEAD e passa il controllo della cpu allo scheduler.
- **Il programma dello scheduler è il seguente:**
- se non detiene il controllo della cpu non fa niente, altrimenti sceglie il processo in stato WAITING con tempo minimo e gli assegna il controllo della cpu.

Esercizio 11

- I programmi di tutti i processi e dello scheduler devono essere eseguiti in parallelo nella main rule;
- nella main rule deve anche essere eseguita una regola che genera nuovi processi:
- un nuovo processo ha una probabilità del 10% di essere generato;
- ogni nuovo processo è in WAITING;
- il tempo di esecuzione deve essere compreso nell'intervallo [1,10].

Bambini e caramelle

Modellare in AsmetaL il seguente problema:

Ci sono 6 bambini seduti a tavola in cerchio.

All'inizio ogni bambino ha 4 caramelle.

Ad ogni passo di simulazione, la macchina sceglie un bambino a caso che esegue una delle seguenti mosse:

- se ha almeno una caramella, ne mangia una;
- se non ha alcuna caramella:
- se il suo vicino a destra ha almeno una caramella, ne ruba una ma non la mangia immediatamente;
- se il suo vicino a destra non ha alcuna caramella, controlla il vicino a sinistra: se questo ha almeno una caramella, ne ruba una e la dà al suo vicino di destra.